

Breaking the Speed Limit

Fast statistical models with Python 3.14, Numba, and JAX

Validate the statistical result first. Then accelerate the bottleneck.

Wenxin Jiang · Jian Yin

Department of Biostatistics, City University of Hong Kong · PyCon US 2026



github.com/LucaJiang/
FastStatisticalModels4Python

1. Why simulation is the statistical test harness

Real biomedical data rarely comes with ground truth. Simulation lets us know what should be recovered, calibrated, or preserved.

Define target recovery / calibration / estimand

Generate scenarios signal, dimension, hard cases

Reference first readable Python oracle

Validate equivalence / recovery / calibration

Scale measure bottlenecks

Governing rule
Same question. Same result. Then faster code.

2. What a statistician has to decide

Question	Decision before timing
Scientific target	estimator, statistic, null model
Ground truth	simulated labels, known effect, or nominal type-I rate
Acceptance criterion	exact match, tolerance, calibration band
Scale stress	N, d, K, p, R, workers, batch size

Reference

readable oracle

Scenario grid

property / stress / load tests

Validation gate

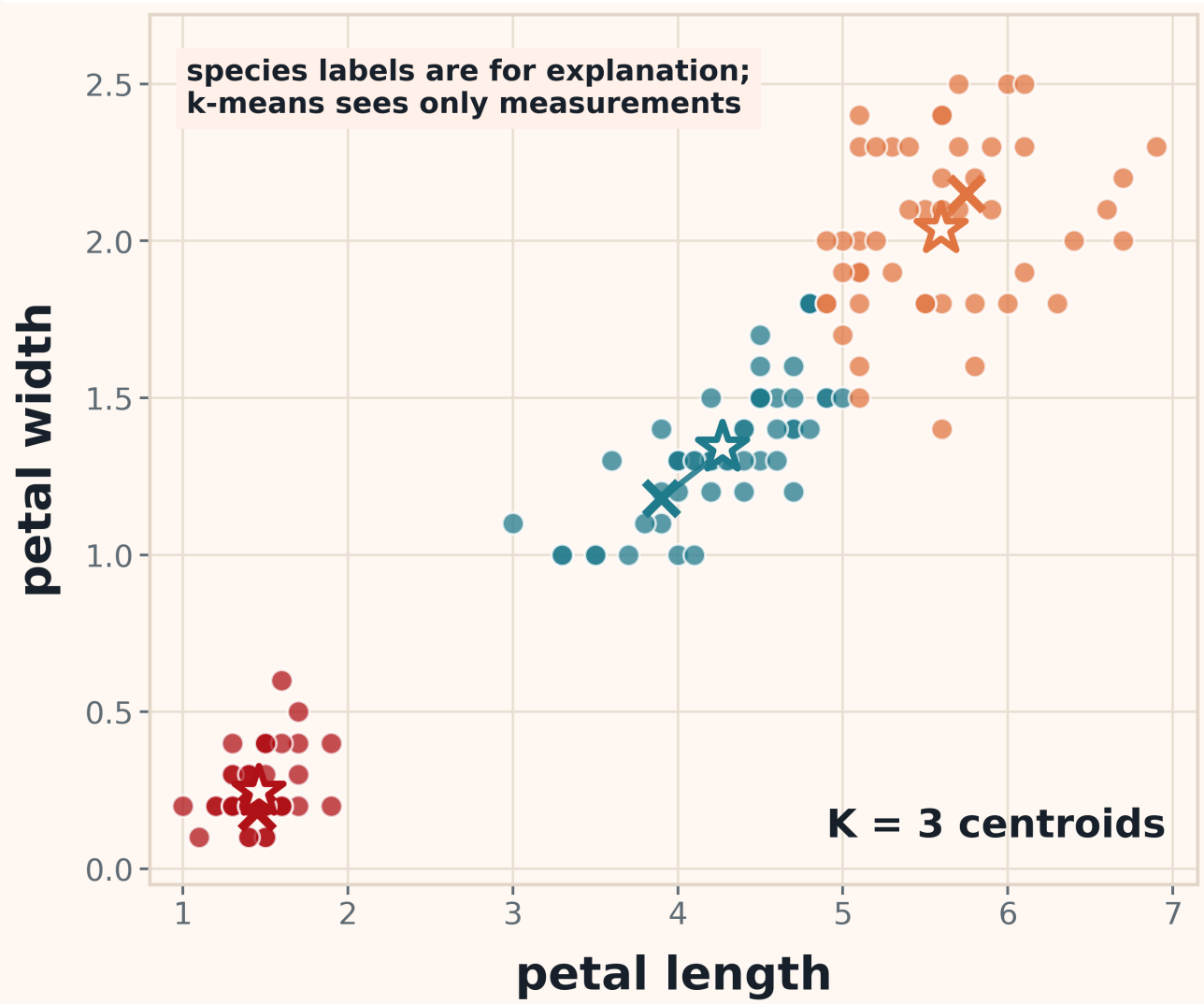
optimize only after checks

3. Two workloads, two pressure shapes

They are simple examples of two common statistical computing patterns.

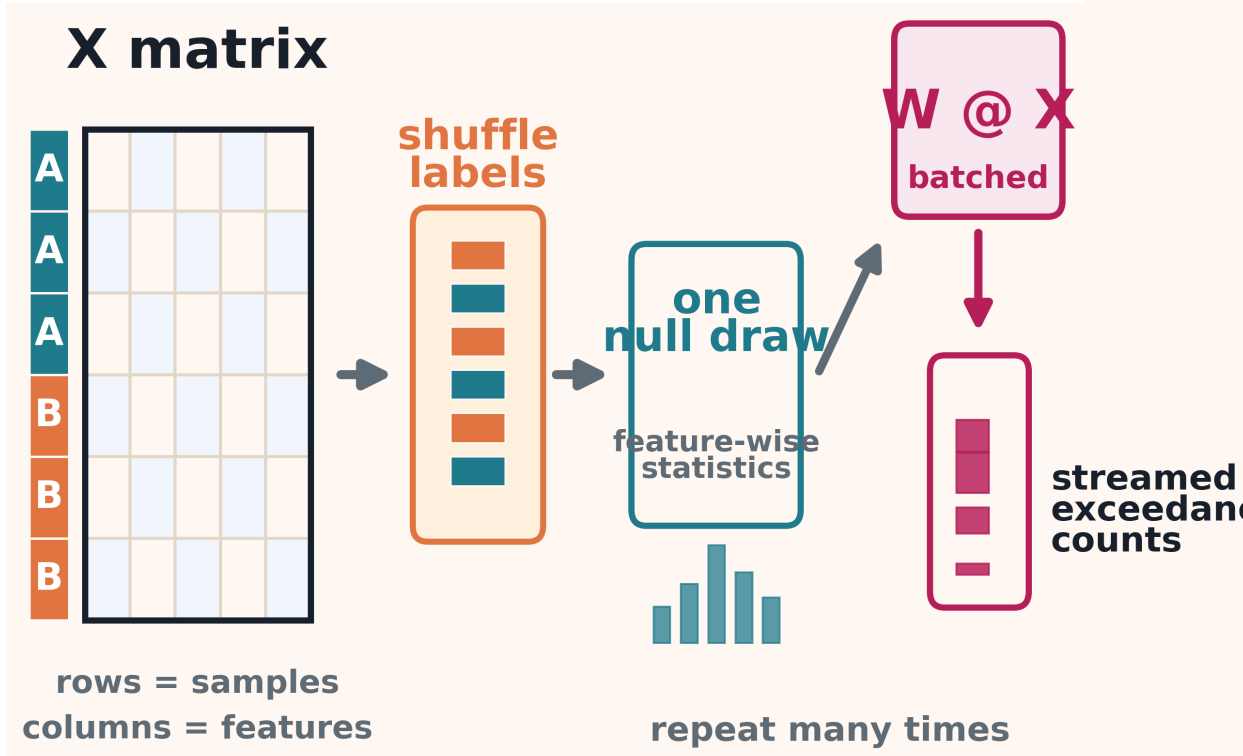
k-means

Iterative model fitting.



permutation test

Resampling inference.



Validation checks

k-means: same data, initialization, stopping rule; compare inertia and recovery.

permutation: same permutation stream and p-value definition; check null calibration before timing.

4. Evidence examples, scoped

Claim	Committed evidence
k-means equivalence	max relative inertia difference 3.1×10^{-14} ; tolerance 10^{-8}
permutation equivalence	max p-value difference 0.0; max statistic difference 9.4×10^{-16}
null behavior	type-I estimate at $\alpha = 0.05$ was 0.051
A100 boundary	first streamed-reduction win at $n = 5,000$, $p = 10,000$, $R = 5,000$, $\text{batch_R}=8,192$
A100 scope	largest slide-level measured speedup 8.54x

Timing scope

speedup = matched CPU matrix baseline / A100 streamed full end-to-end
compile excluded · transfer included · kernel-only excluded

5. Choose the smallest tool

Signal	Conservative tool choice
not trusted yet	fix method / reference
hot scalar loop	Numba
dense algebra	NumPy / BLAS
shared-array repetition	Python 3.14 / threads
device-resident batches	JAX / A100
giant temporary	rewrite / stream

trust first

compile loops

use algebra

batch for device

6. What to report with every speed claim

- validation target and acceptance tolerance
- environment tier: local, server CPU, or A100
- cold vs warm timing; compile excluded or included
- transfer, allocation, and collection included or excluded

Unavailable/OOM cells are reported as unavailable.

Takeaway

1
Simulate known behavior

2
Validate the result

3
Measure the bottleneck

4
Choose the smallest tool

Clear enough to trust. Fast enough to scale.

github.com/LucaJiang/FastStatisticalModels4Python